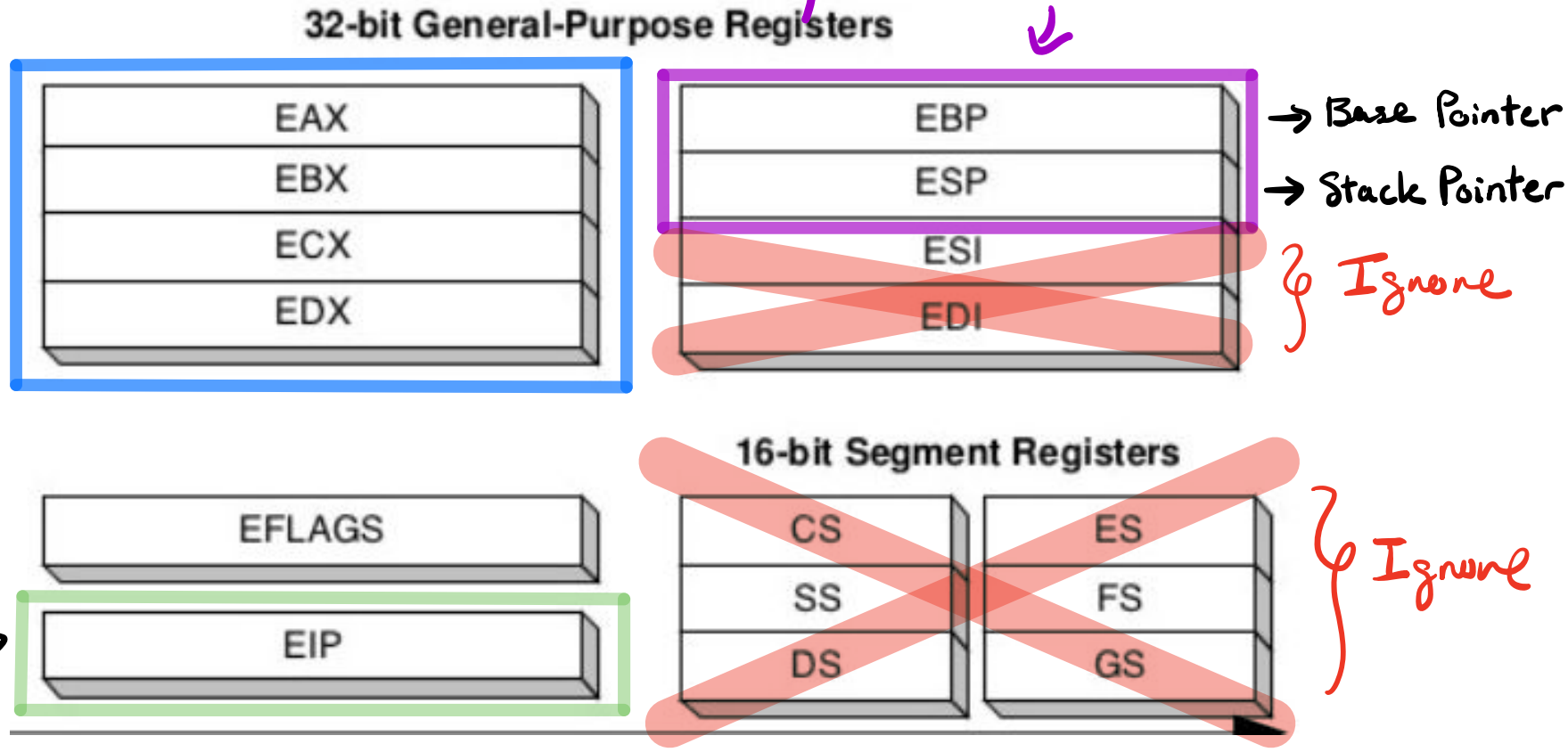
Recitation 12: x86

CSUY 2214

Basics - Architecture

Mainly used for Call Stack

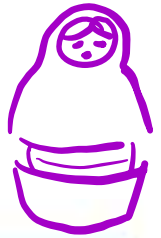
The main regs we will be working with! (Think $1-$7 from E20)

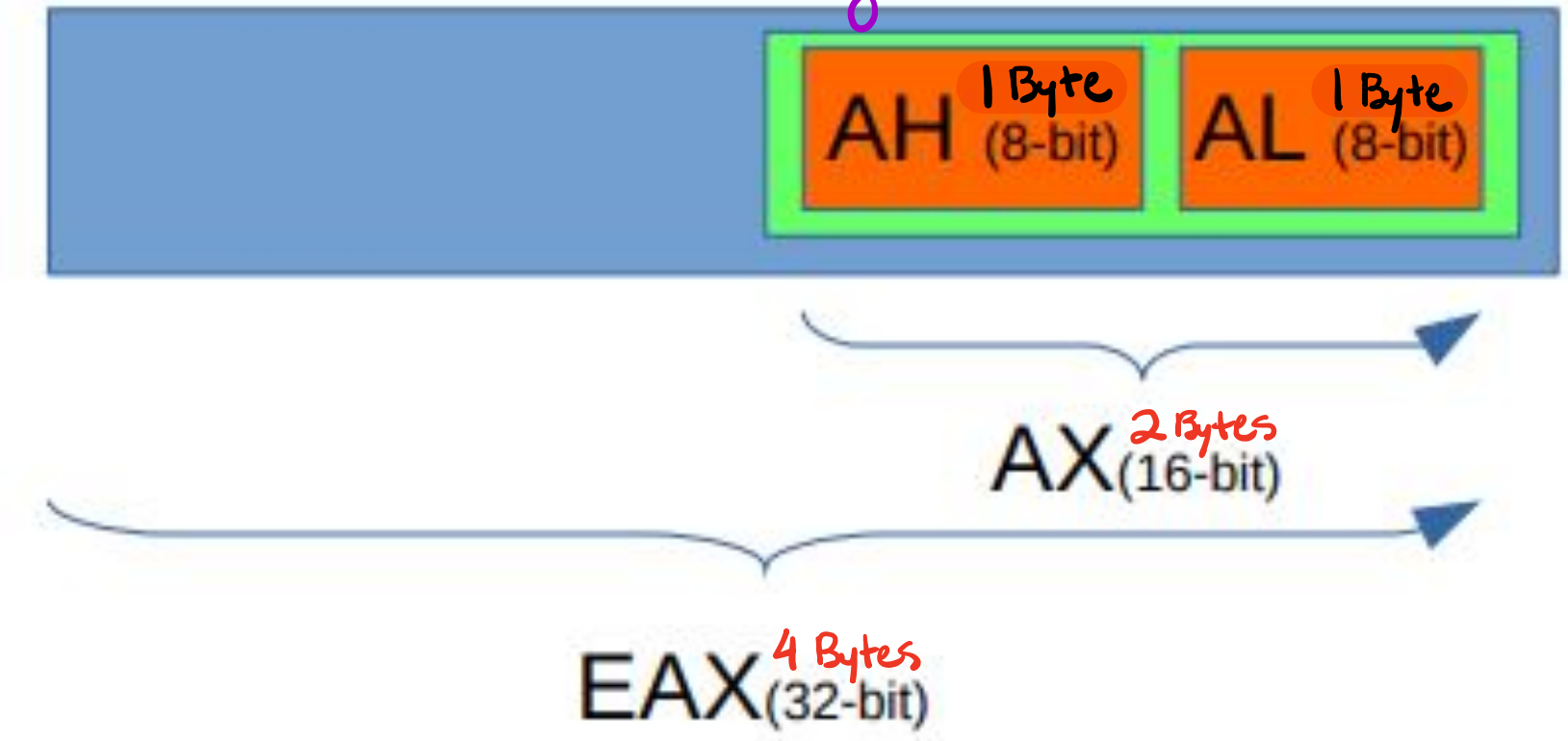


Same as the pc! →

Basics - Architecture

kinda like the Russian Nesting dolls





Jumps and EFLAGS

Instruction	Example	Meaning
Compare	cmp eax, ebx	Set control flags: ZF, CF, OF, SF
Unconditional jump	jmp label	EIP = label
Conditional jump	je label jz label	jump if ZF
	jne label jnz label	jump if !ZF
	jecxz label jcxz label	jump if ECX == 0 jump if CX == 0
	jc label	jump if CF
	jnc label	jump if !CF
Signed comparison	jl label jg label jle label jge label	jump if SF != OF jump if !ZF && SF == OF jump if ZF || SF != OF jump SF == OF
Unsigned comparison	jb label ja label jbe label jae label	jump if CF jump if !CF && !ZF jump if CF || ZF jump if !CF

OF = Overflow Flag
CF = Carry Flag
ZF = Zero Flag
SF = Sign Flag

All arithmetic instrs set EFlags (just like EIS!)

E15 vs E20 vs IA-32

	E15	E20	IA-32 (aka x86)
Memory	Sixteen 12-bit cells of instruction ROM, no data memory	8192 16-bit cells of mixed instruction/data RAM	Up to 4GB of mixed instruction/data RAM
Registers	Four 4-bit registers	Seven general-purpose 16-bit registers	Eight 32-bit general-purpose, eight 80-bit floating point, six 32-bit segment registers, plus various vector, debug, and system registers
Instructions	11 unique instruction mnemonics	13 instructions, 3 psuedo-instructions	About 981 unique instruction mnemonics, considerably more if variations of operand type are taken into account
Real-world use	none	none	Probably in your computer right now

Basics - Immediates

- Write numeric literals in the appropriate base:
  - (dec): 244
  - (bin): 11110100b
  - (hex): 0xf4 or 0f4h → When using 'h' suffix, if # starts w/ a,b,c,d,e,f: Put zero in front
- Labels act the same as e20
  - `my_label: inc eax`

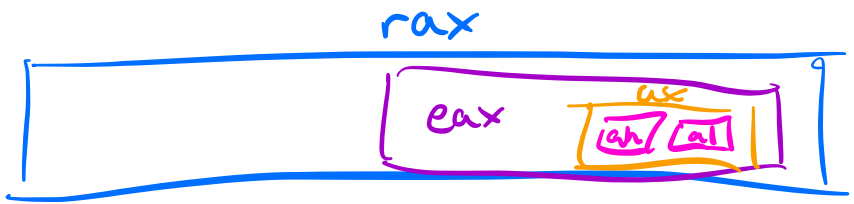
"The value of a label is the memory address where it's defined."
- CompArch Mantra

Basics - Sizes

MUST MEMORIZE!

	size	register examples	directive
byte	8 bits	ah, al, bh, bl, ...	db
word	16 bits (two bytes)	ax, bx, cx, dx, ...	dw
dword Double Word	32 bits (four bytes)	eax, ebx, ecx, edx, ...	dd
qword Quad Word	64 bits (eight bytes)	rax, rbx, rcx, rdx, ...	dq

NOTE: 1 byte = 2 hex digits



Syntax

- Can use `'[]'` to access memory
  - `mov eax, [20h]`
  - `mov [ecx], edx`
- Addresses/Pointers are ***32 bits***
  - ✗ `[bh]` `; invalid` → "bh" is a 8 bit reg.
  - ✓ `[0bh]` `; valid` → "0bh" is an imm written in hex β can be zero-extended to 32-bits.

(equ. to 11)

Invalid Syntax

- Reg sizes are fixed
- Imms can be zero-extended to become larger (but cannot become smaller)

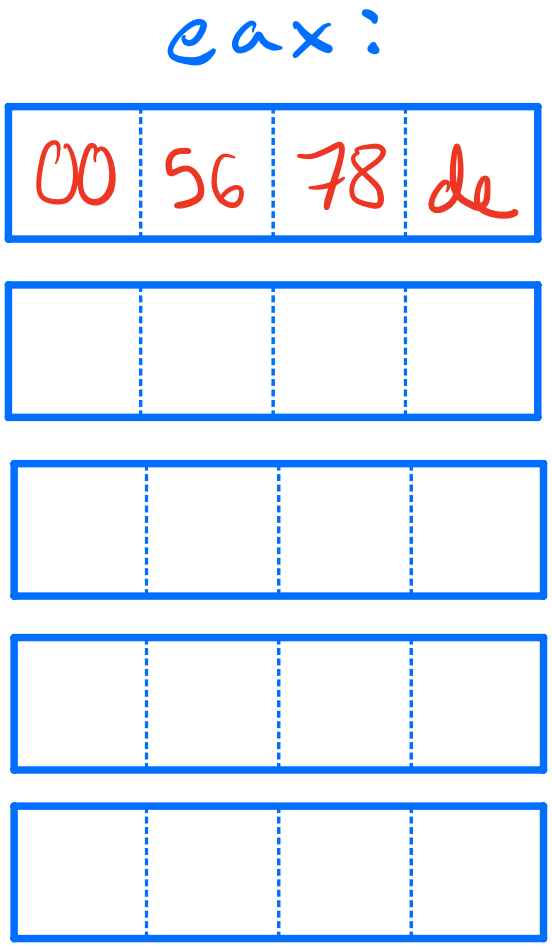
- Operand sizes must match
  - ✗ `mov ah, 0x4932 ; invalid` (ah: 1 byte; 0x4932: (At least) 2 bytes)
  - ✗ `mov ebx, dl ; invalid` (ebx: 4 bytes; dl: 1 byte)
- Only get up to one memory access per instruction
  - ✗ `mov [eax], [ebx] ; invalid`
- If operand size not implicitly declared, must explicitly specify
  - ✗ `mov [label], 53h ; invalid` (Don't know how many bytes to store...)
  - ✓ `mov dword [label], 53h ; valid`

"53h" is stored as a 4 byte value at starting addr label

Happens when we don't have a register operand

Example

```
mov eax, 0x5678de

add ah, 0xff

add ax, 0xff

shl al, 1

shr eax, 4
```

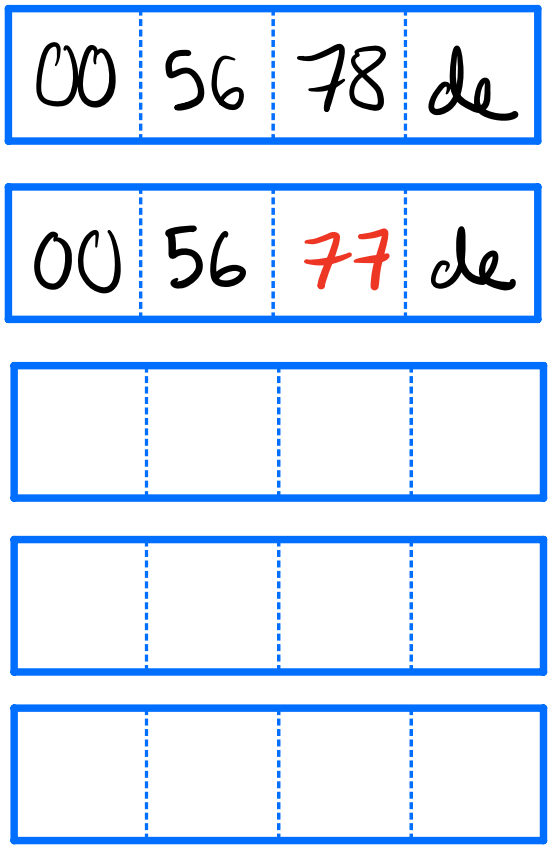


Example

```
mov eax, 0x5678de

add ah, 0xff

add ax, 0xff

shl al, 1

shr eax, 4
```

~~1~~ 1
0x78
+ 0xff
0x77

eax:

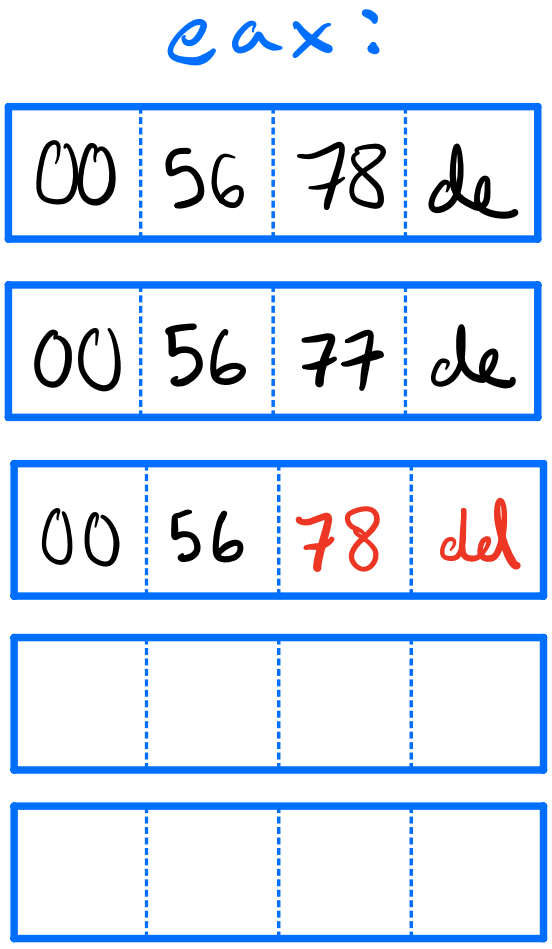


Carry overflows blc of operand size

Example

```
mov eax, 0x5678de

add ah, 0xff

add ax, 0xff

shl al, 1

shr eax, 4
```

$$\begin{array}{r} {\scriptstyle 1\ 1} \\ 0x\ 77de \\ +\ 0x\ 00ff \\ \hline 0x\ 78dd \end{array}$$



Example

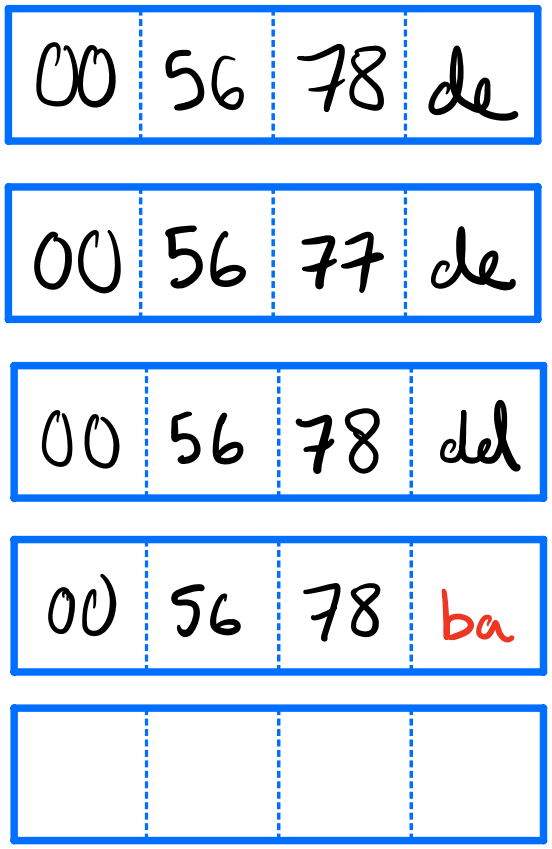
```
mov eax, 0x5678de

add ah, 0xff

add ax, 0xff

shl al, 1

shr eax, 4
```

de = 11011101

de << 1 = 10111010

= ba

eax:



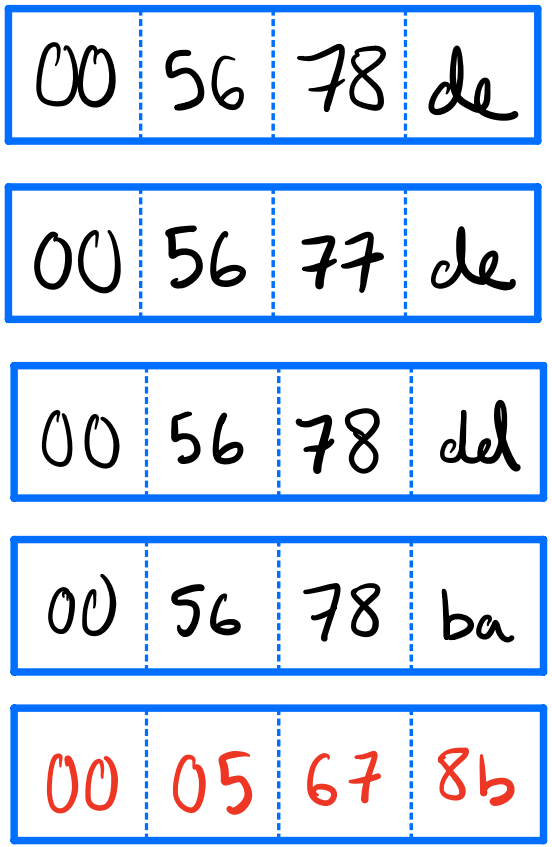
Example

```
mov eax, 0x5678de

add ah, 0xff

add ax, 0xff

shl al, 1

shr eax, 4
```

4 bits = 1 hex digit

005678bex >> 4 = 0005678b

eax:



E20 Cache Simulator - Edge Case

What should happen if a program tries to change the value of $0?

```
lw $0, 1($0)
```

It should happen! ⇒ We set $0 = 0 at the end of each cycle

(4) Not executing this will mess w/ your cache accesses.